

Miniprojet Prolog

Le code est disponible dans le fichier `cube.pl`

Question 1

La situation présente donne:

```
:- dynamic(on/2). % les clauses de "on" peuvent changer
on(a, b).
on(b, c).
on(c, table).
```

Question 2

Préconditions:

- rien sur A
- A différent de table
- A différent de B
- rien sur B ou B = table

Postconditions:

- A sur B

Question 3

L'implémentation de ces conditions donne:

```
put_on_pre(A, B) :-
    \+ on(_, A), % rien sur A
    A \= B, % A != B
    A \= table, % A != table
    (\+ on(_, B); B = table). % rien sur B ou B = table

put_on_post(A, B) :-
    on(A, B). % A sur B
```

Question 4

```
put_on(A, B) :-
    put_on_pre(A, B),
    retract(on(A, _)),
    assertz(on(A, B)),
    put_on_post(A, B).
```

Question 5

On ajoute seulement un `assertz/1` pour enregistrer le mouvement:

```
:- dynamic(move/2). % les clauses de "move" peuvent changer
put_on(A, B) :-
    put_on_pre(A, B),
    retract(on(A, _)),
    assertz(on(A, B)),
    assertz(move(A, B)),
    put_on_post(A, B).
```

Question 6

```
| ?- put_on(a, table).
```

yes

```
| ?- put_on(c,a).
```

no

Question 7

```
clear(A) :-  
    \+ on(_, A). % A déjà libre si rien dessus  
clear(A) :-  
    on(B, A), % B sur A  
    clear(B), % libérer B  
    put_on(B, table). % poser B sur la table  
r_put_on(A, B) :-  
    on(A, B). % rien à faire si A déjà sur B  
r_put_on(A, B) :- % clear les deux avant de mettre A sur B  
    clear(A), % libérer A  
    clear(B), % libérer B  
    put_on(A, B). % placer A sur B
```

Question 8

On travaille récursivement sur la liste des buts:

```
achieve([]). % rien à faire si aucun but  
achieve([H | T]) :-  
    H = on(A, B),  
    r_put_on(A, B),  
    achieve(T). % appel récursif
```

Si on évalue `achieve([on(a,c), on(c,b)])`, on n'a que `on(c,b)`. `achieve/1` a d'abord réalisé `on(a, c)` puis l'a invalidé en réalisant `on(c, b)`.